

EvalCall 方案设计文档（决赛最终版）

自动测试并评估外呼模型在特定任务指令下的指令遵循效果

FINAL	决赛提交版 · 2026-07-14 · 所有数字以仓库正式运行产物为准
-------	--------------------------------------

版本：2026-07-14 赛题：复杂任务指令下的外呼对话模型评测 项目定位：自动测试并评估外呼模型在特定任务指令下的指令遵循效果

1. 执行摘要

EvalCall 面向履约通知、配送改约、骑手招聘、营销外呼和客服回访等任务型外呼场景。系统接收任务指令（SOP）和被测模型或已有对话，通过用户模拟器生成测试对话或直接评估历史日志，将自然语言 SOP 编译为可追溯的 L0/L1 Checklist，逐项评判模型是否遵循任务指令，最终输出模型上线门禁、履约率、P0、关键失败、覆盖率和证据化报告。

外呼模型是唯一被评对象。SOP 是评分依据；用户模拟器是官方要求的测试工具；对话日志是测试过程数据；SOP 诊断、裁判校准、根因归因和人工复核用于保证结论可信，避免把模型、任务指令、裁判或测试数据的问题混为一谈。

2. 问题与设计目标

2.1 业务问题

传统外呼质检依赖人工抽听，成本随话务规模线性增长，反馈慢且标准不统一。普通 LLM 打分又存在四类混杂：模型没有守指令、SOP 自相矛盾、裁判误判、测试数据未覆盖关键分支。只输出总分无法支撑上线决策，也无法指导正确团队修复。

2.2 设计目标

1. 自动测试外呼模型在特定任务指令下的指令遵循效果。
2. 明确交付“可上线 / 打回 / 无法判定”以及履约、P0、关键失败与覆盖率。
3. 通过用户模拟器系统性覆盖 Persona、拒绝、忙碌、隐私和异常分支。
4. 每条判断引用对话证据，并保留 SOP、Checklist、对话、模型和代码版本。
5. 失败后区分外呼模型、SOP/任务指令、裁判链路和测试数据四类根因。

6. 根据根因返回正确步骤，并在同一评分尺下回归。

2.3 非目标

- 不自动修改并发布正式生产 SOP。
- 不把归因结果包装为严格因果证明。
- 不用合成分支冒充真实生产录音或线上全量分布。
- 不以黄金集准确率替代生产持续抽检和人工复核。

3. 产品主流程

3.1 六步语义

步骤	输入	处理	输出
1. 配置测试任务	SOP、目标模型或已有对话、测试模式	标准化任务包、脱敏、版本化	任务包、输入摘要、各输入 hash
2. 建立评分标准	任务包、L0 安全策略	指令编译、来源校验、严重程度分级	L0 通用安全 + L1 业务 Checklist
3. 测试外呼模型	Checklist、目标模型、Persona 或已有日志	模拟对话或日志评估、逐项判定	transcripts、judgments、覆盖与复核信号
4. 输出模型评测报告	全部判定和运行元数据	聚合门禁、履约、P0、覆盖、关键失败	HTML 报告、JSON 汇总、是否上线结论
5. 分析失败原因	模型表现、SOP 体检、裁判与数据质量信号	多指标确定性归纳	target_model / instruction / judge / test_data 根因候选
6. 生成优化与回归计划	根因、证据、版本 hash	按 owner 生成动作与验收条件	模型同尺回归或 SOP 修复后重建评分尺

第四步是产品核心交付终点；第五、六步只在失败后解释并闭环。

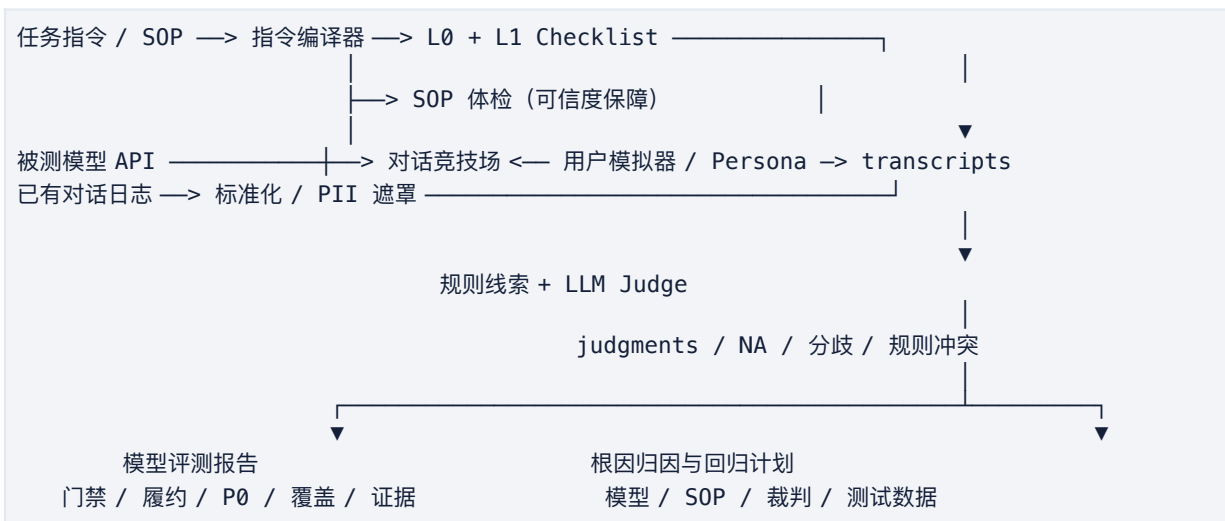
3.2 双测试入口

模拟测试模式：用户模拟器根据 Persona、攻击目标和未覆盖检查点，与被测外呼模型自动生成多轮对话。适合模型上线前系统性补齐边界场景。

已有日志质检模式：导入 JSONL、JSON、CSV、TXT 或 Markdown 对话，标准化后直接评估。适合历史回溯、生产抽检和第三方模型日志接入。

两种入口共享同一 Checklist、Judge、Report 和 Attribution 链路。

4. 系统架构



4.1 核心模块

模块	主要文件	职责
指令编译器	evalcall/compiler.py	将 SOP 编译为 flow、constraint、forbidden、style、outcome、authenticity 检查点，并保留 source_quote
用户模拟器	evalcall/simulator.py	多 Persona、多策略、覆盖驱动和对抗目标生成
对话竞技场	evalcall/arena.py	编排目标模型与模拟器多轮对话，输出统一 transcript
双轨 Judge	evalcall/judge.py、evalcall/rules.py	规则轨提供线索，LLM 轨结合语境逐项判定 pass/fail/NA 并引用证据
安全门禁	evalcall/safety.py	L0 安全与合规红线，P0 一票否决
报告与决策	evalcall/report.py	聚合门禁、履约、P0、关键失败、覆盖率和复核队列
SOP 体检	evalcall/lint.py	检测冲突、不可行、歧义和缺失分支；后端失败时 fail-closed
根因归因	evalcall/attribution.py	综合模型、指令、裁判、数据多类信号，输出根因候选、置信度、owner 和动作
现场服务	evalcall/demo_server.py	六步会话 API、健康检查、样例复用、上传和运行产物管理

5. Checklist 与评判设计

5.1 检查点结构

每个检查点至少包含：id、type、text、source_quote、severity、safety 和策略来源。严重度映射为 P0/P1/P2；其中 P0 失败直接触发整通打回，但报告仍保留其他维度信息。

5.2 规则轨与 LLM 轨

规则轨用于禁语、敏感信息、必要信息等可确定检测，只提供线索，不因关键词命中直接定罪。LLM Judge 结合说话人、上下文和 source_quote 判定 pass/fail/NA，失败必须引用对话证据。规则与 LLM 冲突、NA、高风险或分裂票进入人工复核队列。

5.3 门禁与履约

- 门禁：任一 P0 fail 则该通打回；批次报告同时展示打回通数和原因。

- 履约：由 outcome 检查点判断任务是否实际完成，不被语言流畅度替代。
- 覆盖：统计已触达与未触达检查点，用于发现测试盲区，不单独决定上线。

6. 根因归因算法

归因不能只看全部 judgment 的失败率。当前算法纳入：

1. 通话打回率。
2. 履约率。
3. P0 触发率。
4. 关键流程失败率。
5. critical / major 严重度加权失败率。
6. SOP 可行性分和高严重度体检问题。
7. NA 率、裁判分歧率、规则冲突和复核率。
8. Persona 失败集中度与测试覆盖信号。

当正式输入没有更强的高严重度 SOP 故障证据，而批次出现高打回、低履约或明确 P0 时，系统将外呼模型作为首要根因。若 SOP 存在强冲突或不可行条件，模型归因降级，先由业务 owner 修复任务指令。Scratch lint 迭代不会自动绑定到正式运行，防止探索性草稿污染归因。

归因结果包含置信度和免责声明；低、中置信结论必须人工复核后再修改生产配置。

7. 版本、可审计性与同尺回归

每次正式运行记录：

- SOP、对话、Checklist、安全策略、黄金集和模型版本 hash。
- 代码 commit、裁判模型、被测模型、票数、时间、token 和计价状态。
- 标准化输入、逐项 judgments、summary、manifest、telemetry、review queue 和 HTML 报告。

模型问题回归保持 SOP 和 Checklist hash 不变，返回第三步复测。SOP 修改后必须重新编译 Checklist，并明确说明新旧评分尺不同，不能直接把分数差解释为模型能力提升。

8. 运行模式与故障策略

8.1 已验证产物模式

内置样例在 SOP、完整对话和 Checklist hash 匹配时复用仓库内已验证批次，确保现场演示稳定。页面明确标注“缓存结果·已验证产物”，不会冒充刚刚现场计算。

8.2 本地实时模式

自定义材料通过本地服务调用可用后端。健康检查验证模型后端真实可用性，而不只检查服务进程。后端不可用、超时或输出非法时 fail-closed，返回准确错误，不生成虚假的 100 分健康报告。

8.3 隐私与安全

接入层默认对手机号、身份证、邮箱、银行卡、订单号和地址进行结构化遮罩；遥测不存 prompt 原文。生产环境是否允许数据发送第三方模型由数据 owner 决定，可切换到内网兼容后端。

9. 决赛演示证据

9.1 主演示：配送时间改约

- 正式批次：10 通。
- Checklist：21 项（L0 通用安全 5 项，L1 业务规则 16 项）。
- 逐项判断：210 条。
- 打回：3/10。
- 平均分：62.5。
- 履约率：40.0%。
- P0：3/10。
- 关键流程失败率：25.4%。
- 首要根因：target_model，中置信；模型分 48。
- 限制：NA 9.0%，裁判分歧 0%；归因仍是确定性信号归纳，不是因果证明。
- 回归：只改模型/对话策略，返回第三步；SOP、Checklist 与 10 通用户输入 hash 不变，正常 Judge 重评且不修改 judgments。
- 实际 v2：门禁“可上线”，P0 0/10，履约率 40.0%（不退化），平均分 91.4；原生 diff 为 6 项改善、1 项 style 退化待复测。
- 可比性：instruction_hash、checklist_hash、judge_policy_hash 与用户输入 hash 均一致。

9.2 对照案例：低延迟直播升级

- 正式批次：10 通，9 通打回，履约率 10.0%。
- SOP 可行性：0/100，4 项高严重度问题。
- 首要根因：instruction，高置信。
- 动作：先修 SOP，回到第二步重新编译 Checklist；新旧尺不直接比较模型能力。

9.3 裁判校准

- 32 case，165 个已知真值。
- 整体准确率：92.73% (153/165) 。
- P0 fail recall：11/11。
- style 类型准确率：70%。
- 12 个错判的平均自报置信度：96.25%。

上述数字只适用于指定模型、提示词、票数和黄金集，不外推为线上准确率或零漏判承诺。

10. 测试与验收

项目包含编译、判定、报告、归因、失败分支、后端 fail-closed、Demo API 和六步工作台自动化测试。最终验收必须同时满足：

1. 至少一套样例首要根因为 target_model。
2. 至少一套样例首要根因为 instruction。
3. 模型问题案例第六步返回第三步，SOP 与 Checklist hash 不变。
4. 用户模拟器、Persona、数量、覆盖和未触达检查点在工作台可见。
5. 缓存结果与实时结果分标。
6. 样例输入数、正式报告数和页面数字一致。
7. 自动化测试、接口测试、浏览器测试和材料渲染检查通过。
8. 验收账本和证据索引可追溯到具体产物。

11. 部署与快速开始

本地现场演示：

```
EVALCALL_BACKEND=codex-cli \  
EVALCALL_MODEL=gpt-5.6-sol \  
EVALCALL_REASONING_EFFORT=xhigh \  
EVALCALL_CODEX_TIMEOUT=600 \  
python3 -m evalcall demo --host 127.0.0.1 --port 8765
```

浏览器打开 <http://127.0.0.1:8765/app.html>。公网备用入口为：

<https://kaijie0074-art.github.io/evalcall/app.html>

代码仓库：

<https://github.com/kaijie0074-art/evalcall>

12. 当前边界与后续路线

当前样例规模用于验证方法与产品闭环，不能代表全部线上话务分布。下一阶段将扩展真实脱敏日志、由独立质检维护 sealed holdout、接入真实目标模型 API 进行同一 SOP 下多版本对比，并将门禁接入模型发布流水线。对于高 NA、style 短板和数据分布漂移，持续保留人工复核与再校准机制。

13. 最终交付物

- 可现场操作的六步工作台与公网备用版本。
- GitHub 完整代码、README、当前方案规格和自动化测试。
- 四套独立命名样例及正式运行产物。
- 模型评测报告、根因归因、优化和同尺回归计划。
- 决赛 PPT、10 分钟主讲稿、10 分钟问答预案、一页纸和攻防卡。
- 全过程验收账本与证据索引。